

Constraint-Based University Scheduling Platform

Project Understanding & Design Report — Version 1

This report explains the project from the beginning in simple words: what problem it solves, what the system contains, how scheduling works, what makes the project technically interesting, and what we plan to build step by step.

1. What are we building?

We are building a web application that automatically creates a university timetable. Instead of an administrator manually deciding which course goes into which room, at what time, and with which professor, the system takes all those requirements and finds a valid schedule automatically.

The important part is that the system does not simply create any timetable. It must obey rules such as: a professor cannot teach two classes at once, a room cannot contain two classes at once, a room must be large enough, a lab course needs a suitable lab, and students should not have overlapping classes.

2. A simple example

Suppose a university has 100 courses, 30 professors, 20 classrooms, 5 labs and 8 usable periods per day. There are an enormous number of possible combinations. A human might try to assign courses one by one and repeatedly fix conflicts. Our system instead represents the rules mathematically and asks a constraint solver to find a schedule that satisfies the important rules while also trying to make the timetable convenient.

Course	Professor	Students	Requirement
Database	Prof. A	60	Normal room
Operating Systems	Prof. B	45	Normal room
Networks Lab	Prof. C	30	Computer lab

3. Main things in the system

Component	Purpose
Courses	The classes that need to be scheduled.
Professors	Who teaches each course and when they are available.
Rooms	Classrooms/labs, their capacity and capabilities.
Student groups	Groups/cohorts that must attend particular courses.
Time slots	The periods during which classes can happen.
Constraints	Rules the schedule must follow.
Scheduler	The engine that searches for a good schedule.
Schedule	The final assignment of course + professor + room + time.
Admin dashboard	The interface used to create data, generate schedules and modify them.

4. Hard constraints vs soft constraints

Hard constraints are rules that must never be broken. For example, the same professor cannot teach two courses at the same time. If a generated timetable violates a hard constraint, it is not a valid timetable.

Soft constraints are preferences. They can be violated when necessary, but the system should try to minimize those violations. Examples include avoiding large gaps in a student's day, reducing professor idle time, preferring requested teaching periods, and reducing wasted room capacity.

This distinction is central to the project: hard constraints define what is allowed, while soft constraints define what is desirable.

5. How the scheduler thinks

We will use a constraint-optimization approach rather than a machine-learning model. A solver such as Google OR-Tools CP-SAT can search through possible assignments while respecting the rules.

1. Load courses, professors, rooms, student groups and available times.
2. Create mathematical decision variables representing possible assignments.
3. Add hard constraints that cannot be violated.
4. Add a scoring function for soft constraints.
5. Ask the solver to find a feasible schedule and improve its score.
6. Store the resulting schedule in the database.
7. Return the schedule to the dashboard.

6. Why this is an interesting algorithm problem

If there are many courses, rooms and time periods, the number of possible combinations becomes extremely large. Trying every complete timetable manually is not practical. This type of scheduling problem is computationally difficult. Constraint programming helps by representing the rules and pruning impossible choices instead of blindly testing every final timetable.

We will not claim that our system magically checks every possible permutation in seconds or always proves a mathematically optimal answer. Our measurable goal is to produce valid, high-quality schedules efficiently and report actual benchmark results.

7. The most valuable advanced feature: incremental rescheduling

Imagine a timetable has already been generated and a professor suddenly becomes unavailable on Wednesday from 10 AM to 12 PM. A naive system could regenerate the entire timetable. Our planned system will identify the affected courses, keep unaffected assignments fixed where possible, and re-optimize the affected portion.

The dashboard can then show exactly what changed. For example: one course moved from Wednesday 10 AM / Room 204 to Thursday 11 AM / Room 301, with zero new conflicts. This makes the project much closer to a real operational system.

8. Human-in-the-loop dashboard

The administrator will be able to drag a course from one time/room to another. The backend will validate the requested change. If it is invalid, the system will explain why instead of silently accepting it.

Example: moving Database to Tuesday 10 AM may fail because the assigned professor is unavailable or because the student group already has another class. The UI should display the specific violated constraint.

9. Variable-length classes

Not every class has the same duration. A lecture may take 60 minutes, while a lab may take 3 hours. The system therefore should model time as intervals rather than assuming every event occupies exactly one identical slot. This also allows us to represent transition/buffer time when needed.

10. Proposed architecture

Frontend: React/Next.js + TypeScript for the scheduling dashboard and drag-and-drop timetable.

Backend: Python/FastAPI (or another suitable backend) for APIs, validation and scheduling jobs.

Database: PostgreSQL for persistent university and schedule data.

Solver: Google OR-Tools CP-SAT for constraint optimization.

Async jobs: Redis can be used to handle longer scheduling jobs without blocking normal API requests.

Deployment: Docker and a cloud deployment once the core system is stable.

We will not add technologies just to make the stack look impressive. Each component should solve an actual problem in the system.

11. Suggested data model

Entity	Important fields
Course	id, name, duration, student_group, professor, requirements
Professor	id, name, availability, preferences
Room	id, name, capacity, type, equipment
StudentGroup	id, name, size, required_courses
TimeWindow	start, end, day
Constraint	type, severity, parameters
ScheduleEntry	course, professor, room, start, end

12. End-to-end flow

1. Administrator creates/imports university data.
2. System validates the input data.
3. Administrator selects 'Generate Schedule'.
4. Backend starts a scheduling job.
5. Constraint engine builds the optimization problem.
6. Solver searches for a feasible/high-quality schedule.
7. Result is saved to PostgreSQL.
8. Dashboard displays the timetable and its score.
9. Administrator can inspect conflicts, preferences and utilization.
10. Administrator can drag/drop an event; backend validates the change.
11. If a major constraint changes, incremental rescheduling can update affected events.

13. What makes it resume-worthy

- It is not just CRUD: the core problem requires algorithmic reasoning.
- It combines optimization with practical backend engineering.
- It has a meaningful stateful domain model.
- It can demonstrate concurrency/asynchronous processing.
- It has a real user interface rather than only an API.
- It can be benchmarked with generated datasets.
- It gives strong system-design and algorithm interview discussion points.

14. What we should measure

Instead of inventing business impact, we will measure our own implementation. Useful measurements include scheduling time, number of courses/rooms handled, number of hard-constraint violations (which should be zero for accepted schedules), soft-constraint penalty, room utilization, and the effect of incremental rescheduling.

For example, after implementation we might benchmark datasets of 50, 100, 250 and 500 courses. The final resume bullets will use only numbers that our actual system demonstrates.

15. Development roadmap

Phase	What we build
1. Requirements	Finalize rules, entities and exact scope.
2. Data model	Design PostgreSQL schema and sample university data.
3. Basic solver	Generate a conflict-free timetable using hard constraints.
4. Optimization	Add soft constraints and schedule scoring.
5. Backend	Build APIs and scheduling jobs.

6. Dashboard	Display timetable and schedule details.
7. Manual editing	Drag/drop changes with backend validation.
8. Incremental rescheduling	Re-optimize affected courses after changes.
9. Benchmarking	Generate larger datasets and measure performance.
10. Deployment	Dockerize and deploy the finished system.
11. Resume/interview	Write evidence-based resume bullets and prepare explanations.

16. Important project principles

- Build first, then write the resume claims.
- Understand every algorithm and technology we use.
- Prefer measurable results over marketing language.
- Keep the first version small enough to finish.
- Add advanced features only after the core scheduler works.
- Design the system so that every major engineering decision can be explained in an interview.

17. Our current project definition

Name: Constraint-Based University Scheduling Platform

Primary goal: Automatically generate high-quality, conflict-free university timetables from real constraints.

Core technology: Constraint optimization / CP-SAT, not machine learning.

Key advanced features: soft-constraint optimization, drag-and-drop validation, variable-length intervals, and incremental rescheduling.

Success criterion: A working end-to-end system with measurable scheduling performance and a clear technical story.